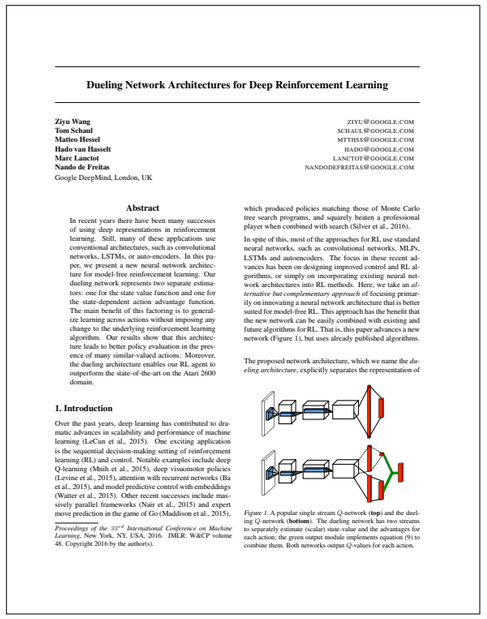
Reference Architecture Figures

(a) Wang 2016 — Dueling DQN

Y-fork V/A streams reference



Dueling Network Architectures for Deep Reinforcement Learning

Ziyu Wang ZIYU@GOOGLE.COM
Tom Schaul SCHAUL@GOOGLE.COM
Matteo Hessel MTTHSS@GOOGLE.COM
Hado van Hasselt HADO@GOOGLE.COM
Marc Lanctot LANCTOT@GOOGLE.COM
Nando de Freitas NANDODEFREITAS@GOOGLE.COM
Google DeepMind, London, UK

Abstract

In recent years there have been many successes of using deep representations in reinforcement learning. Still, many of these applications use conventional architectures, such as convolutional networks, LSTMs, or auto-encoders. In this paper, we present a new neural network architecture for model-free reinforcement learning. Our dueling network represents two separate estimators: one for the state value function and one for the state-dependent action advantage function. The main benefit of this factoring is to generalize learning across actions without imposing any change to the underlying reinforcement learning algorithm. Our results show that this architecture leads to better policy evaluation in the presence of many similar-valued actions. Moreover, the dueling architecture enables our RL agent to outperform the state-of-the-art on the Atari 2600 domain.

1. Introduction

Over the past years, deep learning has contributed to dramatic advances in scalability and performance of machine learning (LeCun et al., 2015). One exciting application is the sequential decision-making setting of reinforcement learning (RL) and control. Notable examples include deep Q-learning (Mnih et al., 2015), deep visuomotor policies (Levine et al., 2015), attention with recurrent networks (Ba et al., 2015), and model predictive control with embeddings (Watter et al., 2015). Other recent successes include massively parallel frameworks (Nair et al., 2015) and expert move prediction in the game of Go (Maddison et al., 2015), which produced policies matching those of Monte Carlo tree search programs, and squarely beaten a professional player when combined with search (Silver et al., 2016).

In spite of this, most of the approaches for RL use standard neural networks, such as convolutional networks, MLPs, LSTMs and autoencoders. The focus in these recent advances has been on designing improved control and RL algorithms, or simply on incorporating existing neural network architectures into RL methods. Here, we take an *alternative but complementary approach* of focusing primarily on innovating a neural network architecture that is better suited for model-free RL. This approach has the benefit that the new network can be easily combined with existing and future algorithms for RL. That is, this paper advances a new network (Figure 1), but uses already published algorithms.

The proposed network architecture, which we name the *dueling architecture*, explicitly separates the representation of

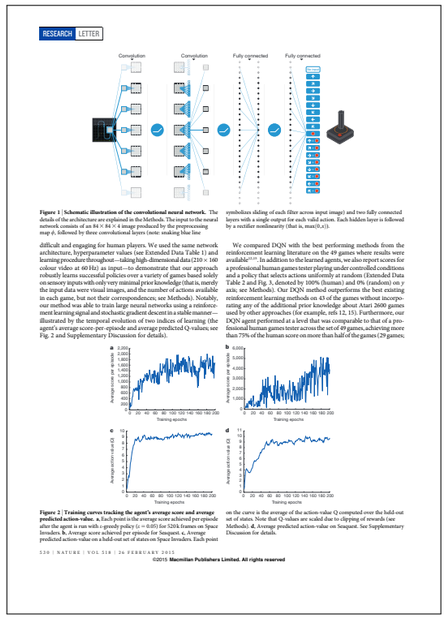
Figure 1. A popular single stream Q-network (**top**) and the dueling Q-network (**bottom**). The dueling network has two streams to separately estimate (scalar) state-value and the advantages for each action; the green output module implements equation (9) to combine them. Both networks output Q-values for each action.

Proceedings of the 33rd International Conference on Machine Learning, New York, NY, USA, 2016. JMLR: W&CP volume 48.

Copyright 2016 by the author(s).

(b) Mnih 2015 — Nature DQN

CNN pipeline with feature maps (p.530)



RESEARCH LETTER

Figure 1 | Schematic illustration of the convolutional neural network. The details of the architecture are explained in the Methods. The input to the neural network consists of an $84 \times 84 \times 4$ image produced by the preprocessing map ϕ, followed by three convolutional layers (note: stealing blue line

difficult and engaging for human players. We used the same network architecture, hyperparameter values (see Extended Data Table 1) and learning procedure throughout—taking high-dimensional data (210×160 colour video at 60 Hz) as input—to demonstrate that our approach robustly learns successful policies over a variety of games based solely on sensory inputs with only very minimal prior knowledge (that is, merely the input data were visual images, and the number of actions available in each game, but not their correspondences; see Methods). Notably, our method was able to train large neural networks using a reinforcement learning signal and stochastic gradient descent in a stable manner—illustrated by the temporal evolution of two indices of learning (the agent's average score-per-episode and average predicted Q-values; see Fig. 2 and Supplementary Discussion for details).

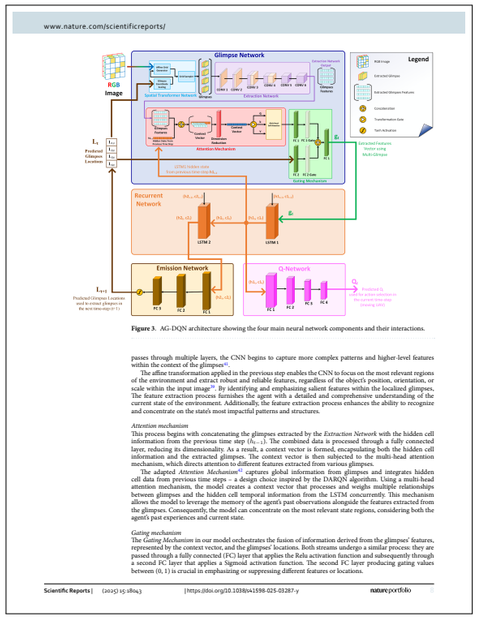
Figure 2 | Training curves tracking the agent's average score and average predicted action-value. a, Each point is the average score achieved per episode after the agent is run with ε-greedy policy ($\varepsilon = 0.05$) for 520k frames on Space Invaders. **b,** Average score achieved per episode for Seaquest. **c,** Average predicted action-value on a held-out set of states on Space Invaders. Each point

on the curve is the average of the action-value Q computed over the held-out set of states. Note that Q-values are scaled due to clipping of rewards (see Methods). **d,** Average predicted action-value on Seaquest. See Supplementary Discussion for details.

©2015 Macmillan Publishers Limited. All rights reserved

(c) AG-DQN 2025 — Scientific Reports

Multi-component colour regions (Fig. 3)



www.nature.com/scientificreports/

Figure 3. AG-DQN architecture showing the four main neural network components and their interactions.

passes through multiple layers, the CNN begins to capture more complex patterns and higher-level features within the context of the glimpses[36].

The affine transformation applied in the previous step enables the CNN to focus on the most relevant regions of the environment and extract robust and reliable features, regardless of the object's position, orientation, or scale within the input image[38]. By identifying and emphasizing salient features within the localized glimpses, The feature extraction process furnishes the agent with a detailed and comprehensive understanding of the current state of the environment. Additionally, the feature extraction process enhances the ability to recognize and concentrate on the state's most impactful patterns and structures.

Attention mechanism

This process begins with concatenating the glimpses extracted by the *Extraction Network* with the hidden cell information from the previous time step (h_{t-1}). The combined data is processed through a fully connected layer, reducing its dimensionality. As a result, a context vector is formed, encapsulating both the hidden cell information and the extracted glimpses. The context vector is then subjected to the multi-head attention mechanism, which directs attention to different features extracted from various glimpses.

The adapted *Attention Mechanism*[42] captures global information from glimpses and integrates hidden cell data from previous time steps - a design choice inspired by the DARQN algorithm. Using a multi-head attention mechanism, the model creates a context vector that processes and weighs multiple relationships between glimpses and the hidden cell temporal information from the LSTM concurrently. This mechanism allows the model to leverage the memory of the agent's past observations alongside the features extracted from the glimpses. Consequently, the model can concentrate on the most relevant state regions, considering both the agent's past experiences and current state.

Gating mechanism

The *Gating Mechanism* in our model orchestrates the fusion of information derived from the glimpses' features, represented by the context vector, and the glimpses' locations. Both streams undergo a similar process: they are passed through a fully connected (FC) layer that applies the Relu activation function and subsequently through a second FC layer that applies a Sigmoid activation function. The second FC layer producing gating values between (0, 1) is crucial in emphasizing or suppressing different features or locations.

(d) MHA-DQN 2026 — Electronics

Pipeline flow + 4 MHA heads (Fig. 2)

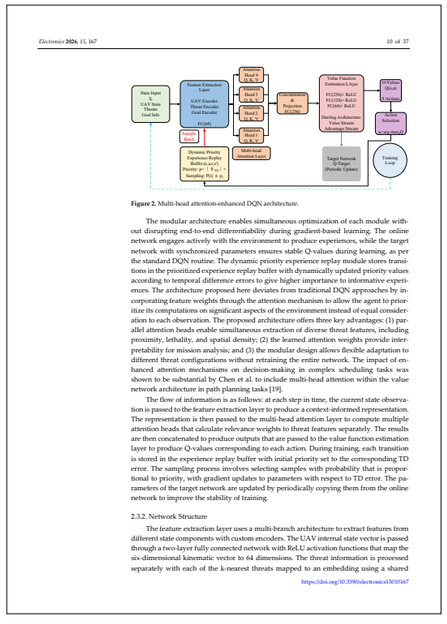


Figure 2. Multi-head attention-enhanced DQN architecture.

The modular architecture enables simultaneous optimization of each module without disrupting end-to-end differentiability during gradient-based learning. The online network engages actively with the environment to produce experiences, while the target network with synchronized parameters ensures stable Q-values during learning, as per the standard DQN routine. The dynamic priority experience replay module stores transitions in the prioritized experience replay buffer with dynamically updated priority values according to temporal difference errors to give higher importance to informative experiences. The architecture proposed here deviates from traditional DQN approaches by incorporating feature weights through the attention mechanism to allow the agent to prioritize its computations on significant aspects of the environment instead of equal consideration to each observation. The proposed architecture offers three key advantages: (1) parallel attention heads enable simultaneous extraction of diverse threat features, including proximity, lethality, and spatial density; (2) the learned attention weights provide interpretability for mission analysis; and (3) the modular design allows flexible adaptation to different threat configurations without retraining the entire network. The impact of enhanced attention mechanisms on decision-making in complex scheduling tasks was shown to be substantial by Chen et al. to include multi-head attention within the value network architecture in path planning tasks [19].

The flow of information is as follows: at each step in time, the current state observation is passed to the feature extraction layer to produce a context-informed representation. The representation is then passed to the multi-head attention layer to compute multiple attention heads that calculate relevance weights to threat features separately. The results are then concatenated to produce outputs that are passed to the value function estimation layer to produce Q-values corresponding to each action. During training, each transition is stored in the experience replay buffer with initial priority set to the corresponding TD error. The sampling process involves selecting samples with probability that is proportional to priority, with gradient updates to parameters with respect to TD error. The parameters of the target network are updated by periodically copying them from the online network to improve the stability of training.

2.3.2. Network Structure

The feature extraction layer uses a multi-branch architecture to extract features from different state components with custom encoders. The UAV internal state vector is passed through a two-layer fully connected network with ReLU activation functions that map the six-dimensional kinematic vector to 64 dimensions. The threat information is processed separately with each of the k-nearest threats mapped to an embedding using a shared

https://doi.org/10.3390/electronics15010167

Old (v1) vs. New (v2)

OLD (v1)

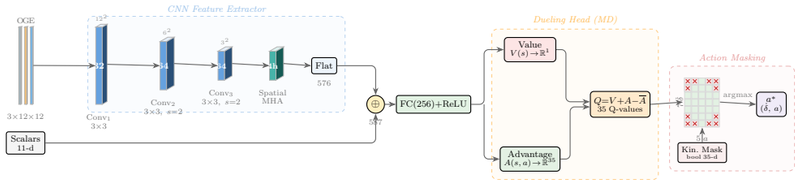
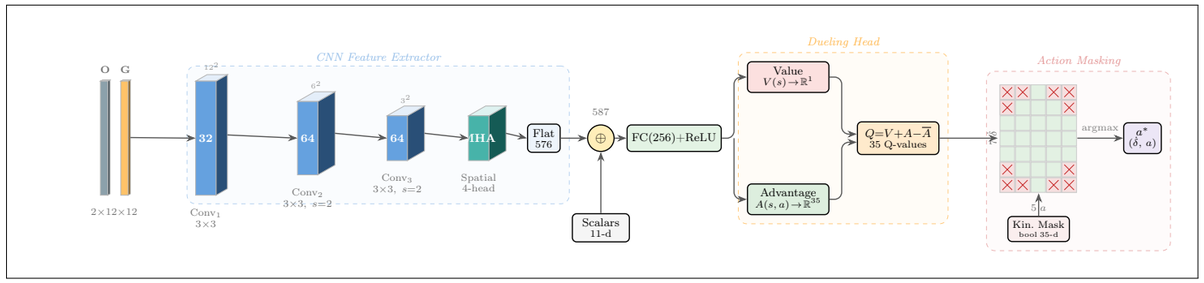


Figure 1: Architecture of the CNN-DDQN+MD Q-network. A convolutional branch extracts spatial features from three 12×12 map channels (occupancy, goal distance field, EDT clearance) through three convolution layers with ReLU activations and spatial multi-head attention (MHA), yielding a 576-dimensional feature vector. This vector is concatenated with 11 scalar state features (pose, velocity, steering angle, goal bearing) and processed by a shared FC layer before splitting into a value stream $V(s)$ and an advantage stream $A(s, a)$ following the dueling architecture. The head produces 35 joint Q-values (7 steering rates $\times$ 5 accelerations), and a kinematic admissibility mask zeroes infeasible actions before argmax policy extraction.

1

Cuboids paper-thin · grid invisible · left-heavy · V/A too far apart

NEW (v2)



Bigger cuboids · visible grid · balanced layout · compact dueling fork